

Sequential logic circuits

- Depends on combination of present inputs & previous inputs.

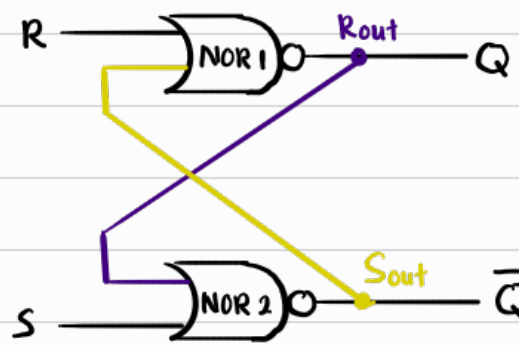
[Follows a sequence of input to get an output]

Combinational vs Sequential



Combinational logic	Aspect	Sequential logic
Only present input	Output depends on?	Present and past input
Fast	Speed from input to output	Most likely slower
Easier	Design process	Usually more difficult
Mostly no	Feedback path from output to input	Yes
No	Dependent on time?	Yes
Logic gates	Elementary building blocks	Flip-flops
Arithmetic and Boolean operators	Mainly used for	Storing data and complicated process
No	Capability of storing states / Memory?	Yes
No, No	Clock? Trigger?	Yes, Yes
Encoder, Decoder, Multiplexer, Demultiplexer	Examples	Counters, Registers, Clocks,

Example 1 [SR Latch (NOR)]



$$R_{out} = Q$$

$$S_{out} = \bar{Q}$$

Assume initially $Q = 1, \bar{Q} = 0$

$\Rightarrow R_{out} = 1, S_{out} = 0$

When $S = 0, R = 0$

1) NOR 1 [$R = 0, S_{out} = 0$] $\Rightarrow Q = 1$

2) NOR 2 [$S = 0, R_{out} = 1$] $\Rightarrow \bar{Q} = 0$

Hold Action

When $S = 0, R = 0$
 $\Rightarrow Q \& R$ doesn't change

This means R_{out} & S_{out} remains as 1 & 0 respectively

• This means the inputs have stabilised
 $\therefore$ Output have stabilised

Assume initially $Q = 0, \bar{Q} = 1$

$\Rightarrow R_{out} = 0, S_{out} = 1$

When $S = 0, R = 0$

1) NOR 1 [$R = 0, S_{out} = 1$] $\Rightarrow Q = 0$

2) NOR 2 [$S = 0, R_{out} = 0$] $\Rightarrow \bar{Q} = 1$

This means R_{out} & S_{out} remains as 0 & 1 respectively

• This means the inputs have stabilised
 $\therefore$ Output have stabilised

When $S = 0, R = 1$

1) NOR 1 [$R = 1, S_{out} = 0$] $\Rightarrow Q = 0$

2) NOR 2 [$S = 0, R_{out} = 1$] $\Rightarrow \bar{Q} = 0$

R_{out} changed to 0

• Inputs changed, not stable
• Output have not stabilised, so have to go another cycle w $S_{out} = 0, R_{out} = 0$

3) NOR 1 [$R = 1, S_{out} = 0$] $\Rightarrow Q = 0$

4) NOR 2 [$S = 0, R_{out} = 0$] $\Rightarrow \bar{Q} = 1$

S_{out} changed to 1

5) NOR 1 [$R = 1, S_{out} = 1$] $\Rightarrow Q = 0$

6) NOR 2 [$S = 0, R_{out} = 0$] $\Rightarrow \bar{Q} = 1$

S_{out} & R_{out} unchanged
 $\therefore$ Stabilised

When $S = 0, R = 1$

1) NOR 1 [$R = 1, S_{out} = 1$] $\Rightarrow Q = 0$

2) NOR 2 [$S = 0, R_{out} = 0$] $\Rightarrow \bar{Q} = 1$

S_{out} & R_{out} unchanged
 $\therefore$ Stabilised

This is called a **RESET**, because Q is forced to be reset to 0 no matter what it was previously

Using the same approach, you will notice ...

When $S = 1, R = 0 \implies Q = 1 \text{ \& } \bar{Q} = 0$

$\therefore$ This is called **SET**, because Q is forced to be set to 1 no matter what it was previously

When $S = 1, R = 1 \implies Q = 0, \bar{Q} = 0$

This is a **forbidden** because it contradicts & defies the output that have been defined to be Q & $\bar{Q}$

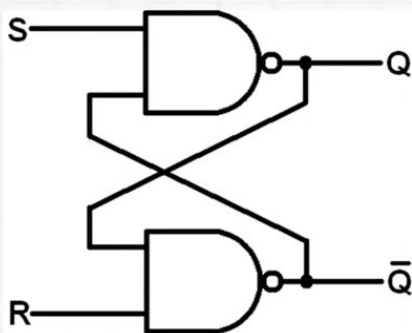
Truth Table

S	R	Q	Q'
0	0	Q	Q'
0	1	0 RESET	1
1	0	1 SET	0
1	1	0 X FORBIDDEN	X 0

Example 2 [SR Latch(Nand)]

Using the same approach as Example 1, we get this

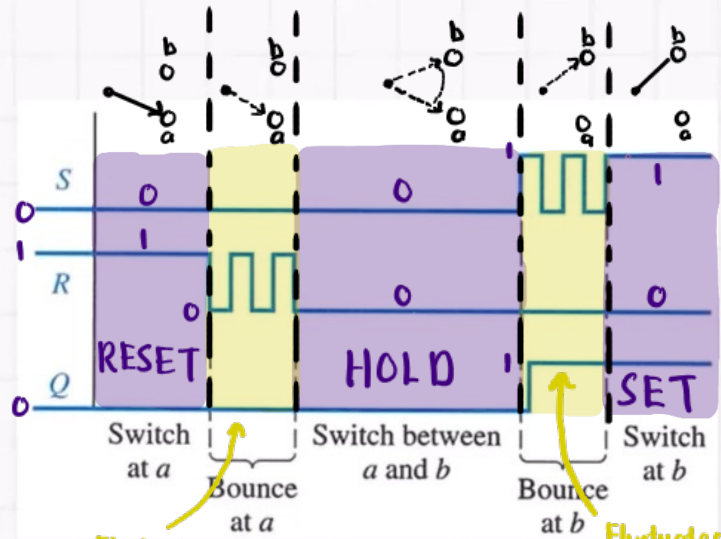
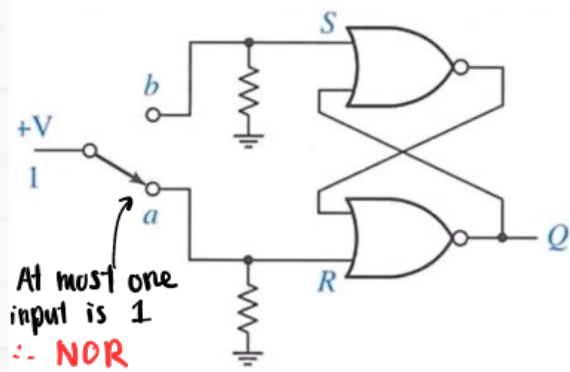
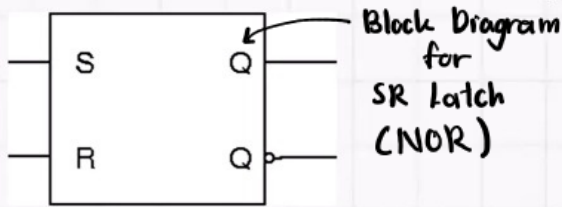
SR Latch (NAND)



S	R	Q	Q'
0	0	X FORBIDDEN	X
0	1	1 SET	0
1	0	0 RESET	1
1	1	Q	Q'

Debouncing w SR Latch

Debouncing switch



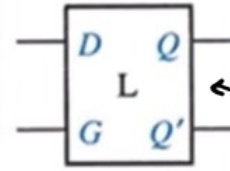
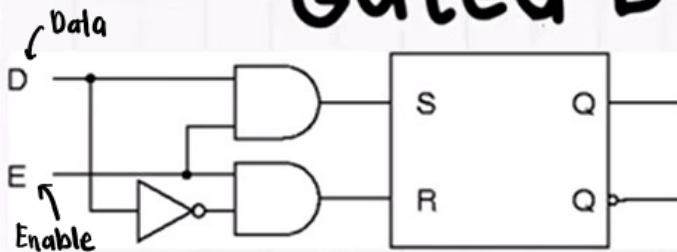
S	R	Q	Q'
0	0	Q	Q'
0	1	0	1
1	0	1	0
1	1	X	X

Fluctuates bwn RESET & HOLD. Since HOLD just maintains the previous output, Q doesn't fluctuate

Fluctuates bwn SET & HOLD. Since HOLD just maintains the previous output, Q doesn't fluctuate

Example 3 [Gated D-Latch]

Gated D-Latch



Block Diagram for Gated D-latch

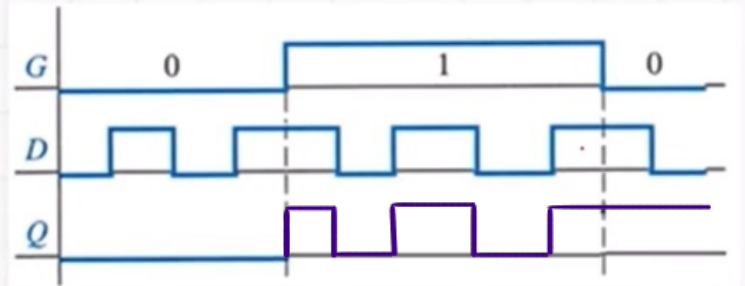
E	D	Q	Q ⁺
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

When Enable is off, Q maintain previous output

Q \ GD	00	01	11	10
0	0	0	1	0
1	1	1	1	0

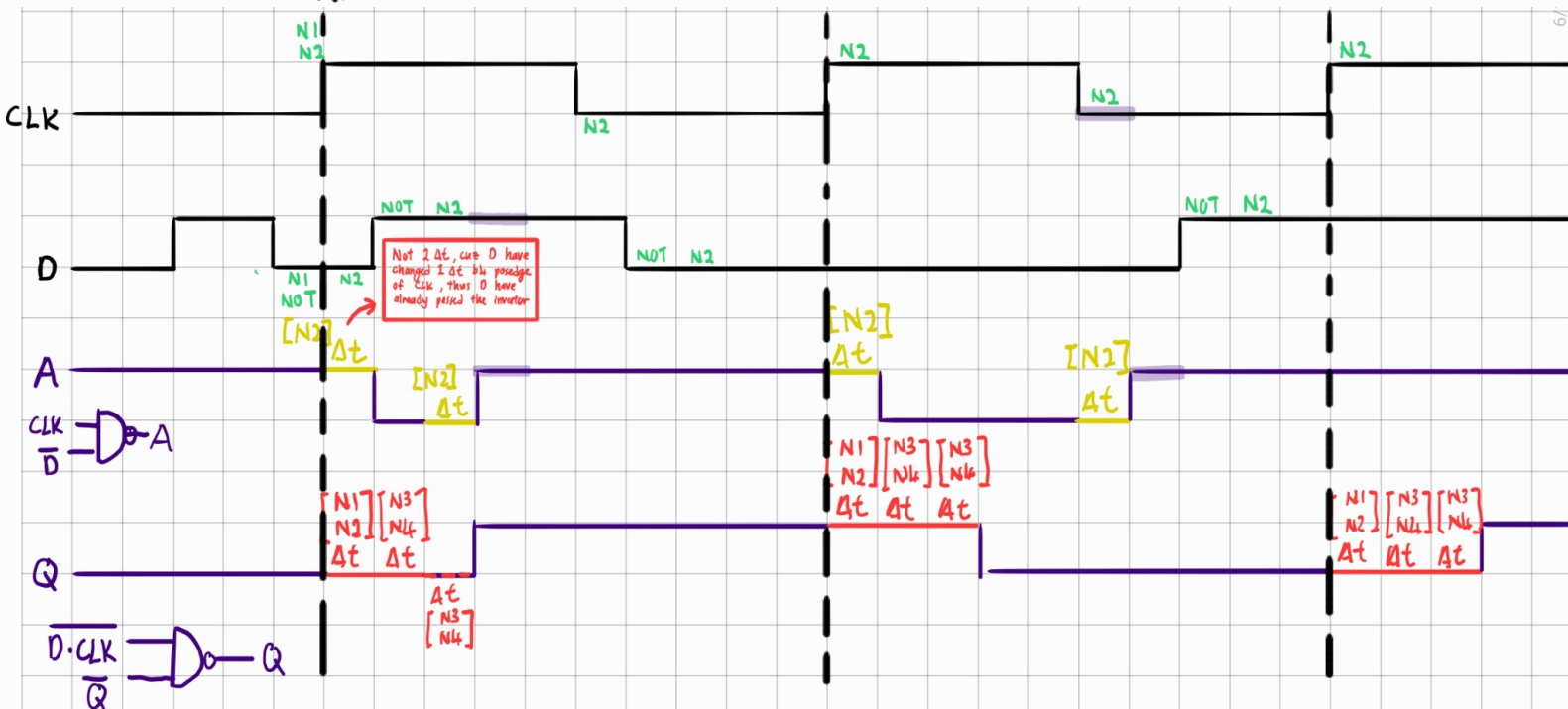
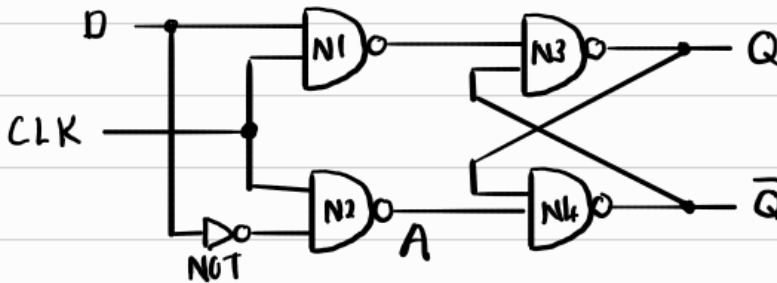
$$Q^+ = G'Q + GD$$

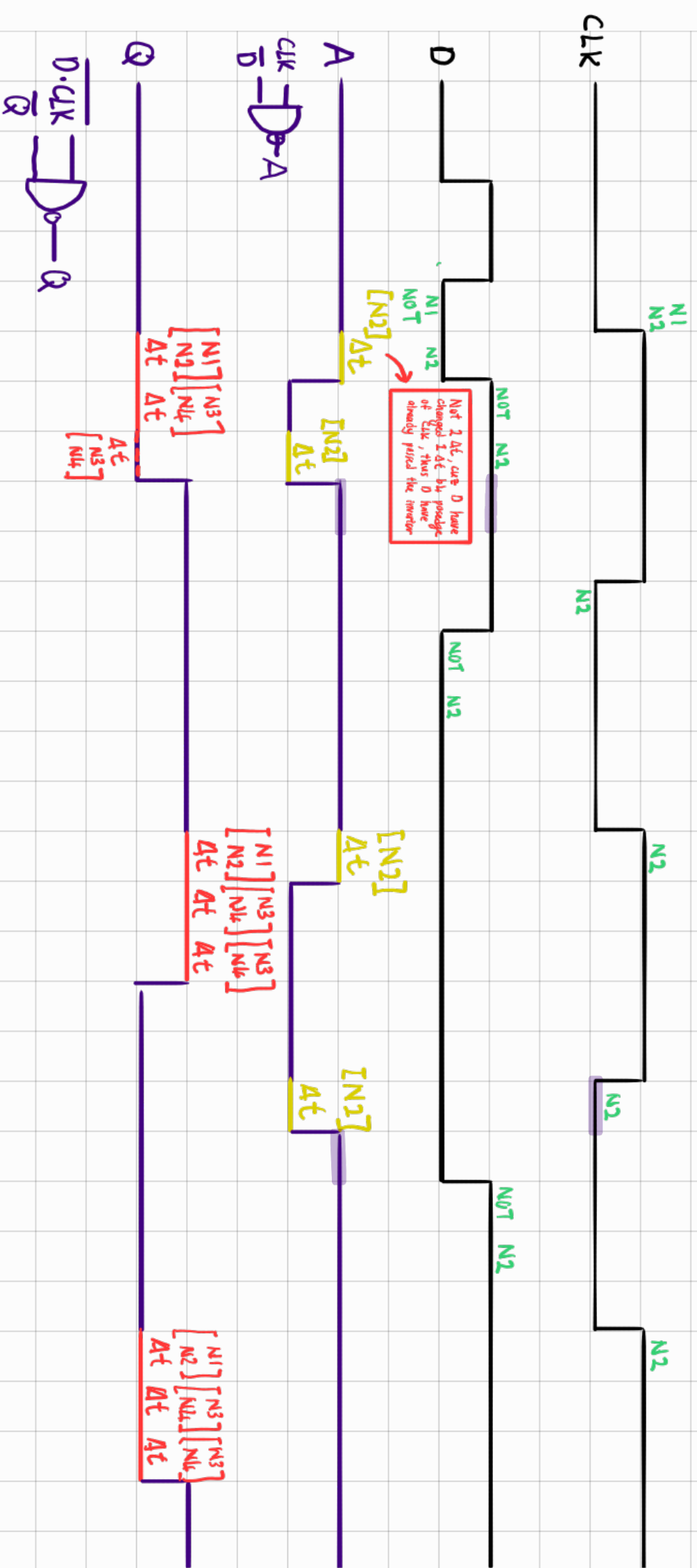
When Enable is ON, Q will output Data



Example

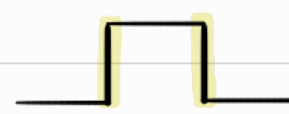
Assume propagation delay thru NAND gate or inverter is Δt . Complete timing diagram





Flip - Flop

LATCH	FLIP FLOP
Building blocks of sequential circuits, built using logic gates	Building blocks of sequential circuits, built using latches
Checks input continuously and changes output correspondingly	Checks input continuously and changes the output in a corresponding manner only with the clock signal
Sensitive to the input switch and is capable of transmitting data as long as the switch is in an on state	Sensitive to the clock signal, and the output will not change until a change occurs in the input clock signal
Work with only binary inputs	Works with binary inputs as well as the clock signal
Level triggered as the output will only change with the change in the binary level	Edge triggered as the output will change based on the positive edge or the negative edge of the clock signal
Cannot be used as a register	Can be used as a register
Asynchronous	Synchronous
No clock signal	Has a clock signal



Flip - Flop

Edge Triggered

Input Configuration

Q = Current Output (B4 CLK Edge)
 Q^+ = Next Output (After CLK Edge)

SET / RESET [SR]

Default: NOR

Improved SR [JK]

Data [D]

Toggle [T]

At pos-edge

At pos-edge

At pos-edge

At pos-edge

	S	R	Q^+
HOLD	0	0	Q
SET	1	0	1
RESET	0	1	0
Invalid	1	1	X

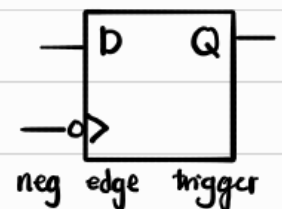
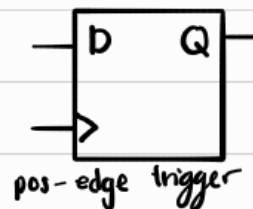
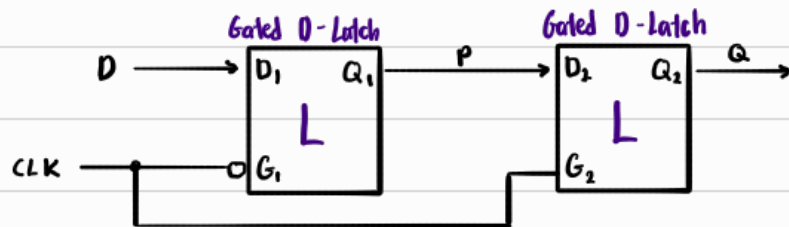
	J	K	Q^+
HOLD	0	0	Q
SET	1	0	1
RESET	0	1	0
Toggle	1	1	Q'

	D	Q^+
RESET	0	0
SET	1	1

	T	Q^+
HOLD	0	Q
Toggle	1	Q'

D Flip-Flop [Master - Slave]

~ Implementation of a flip-flop using 2 latches

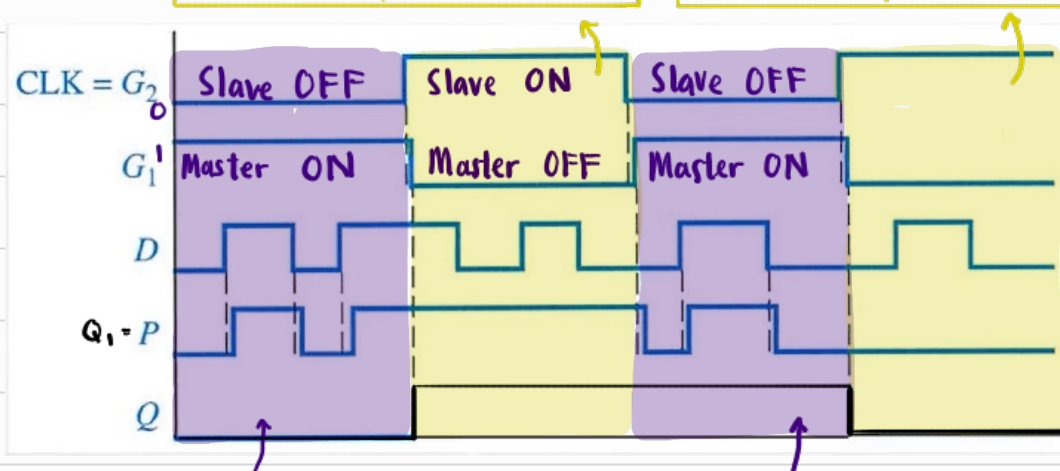


Master OFF, Slave ON

- $P \rightarrow D_2 \rightarrow Q_2 \rightarrow Q$
- ↳ Since Master OFF, P maintains its latest value [1]
- ∴ $P \rightarrow Q$, P maintain

Master OFF, Slave ON

- $P \rightarrow D_2 \rightarrow Q_2 \rightarrow Q$
- ↳ Since Master OFF, P maintains its latest value [0]
- ∴ $P \rightarrow Q$, P maintain



Master ON, Slave OFF

- Data [D] $\rightarrow D_1 \rightarrow Q_1 \rightarrow P$
- ↳ Since Slave OFF, P can't be transmitted to Q
- ↳ Q maintains its latest value [0]
- ∴ $D \rightarrow P$, Q maintain

Master ON, Slave OFF

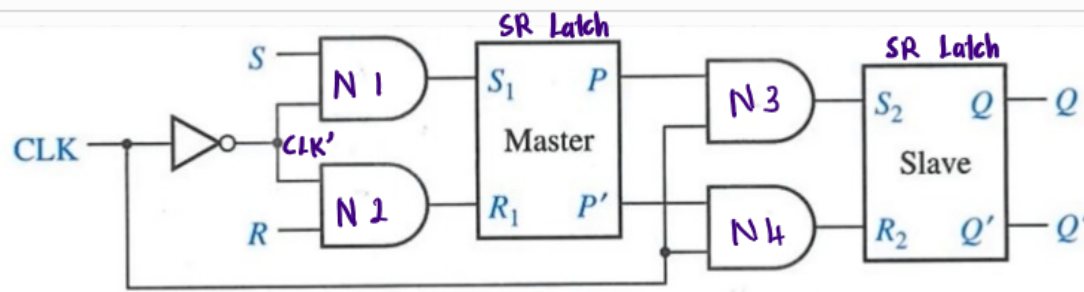
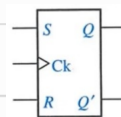
- Data [D] $\rightarrow D_1 \rightarrow Q_1 \rightarrow P$
- ↳ Since Slave OFF, P can't be transmitted to Q
- ↳ Q maintains its latest value [1]
- ∴ $D \rightarrow P$, Q maintain

* Take Note : In this case is the positive Edge of the CLK [Posaedge]

Output [Q] only changes at CLK Edge. In other words Output only changes when CLK changes. This makes the configuration of latches act as a flip-flop, aka. Master - Slave

To make it negative-edge trigger, we just have to move the NOT gate to the Slave gate, so that a CLK change of $1 \rightarrow 0$, causes Slave to go $0 \rightarrow 1$, outputting / updating Q

SR Flip-Flop [Master - Slave]



SR Latch

S	R	Q	Q'
0	0	HOLD Q	Q'
0	1	RESET 0	1
1	0	SET 1	0
1	1	FORBIDDEN X	X

Master OFF, Slave ON

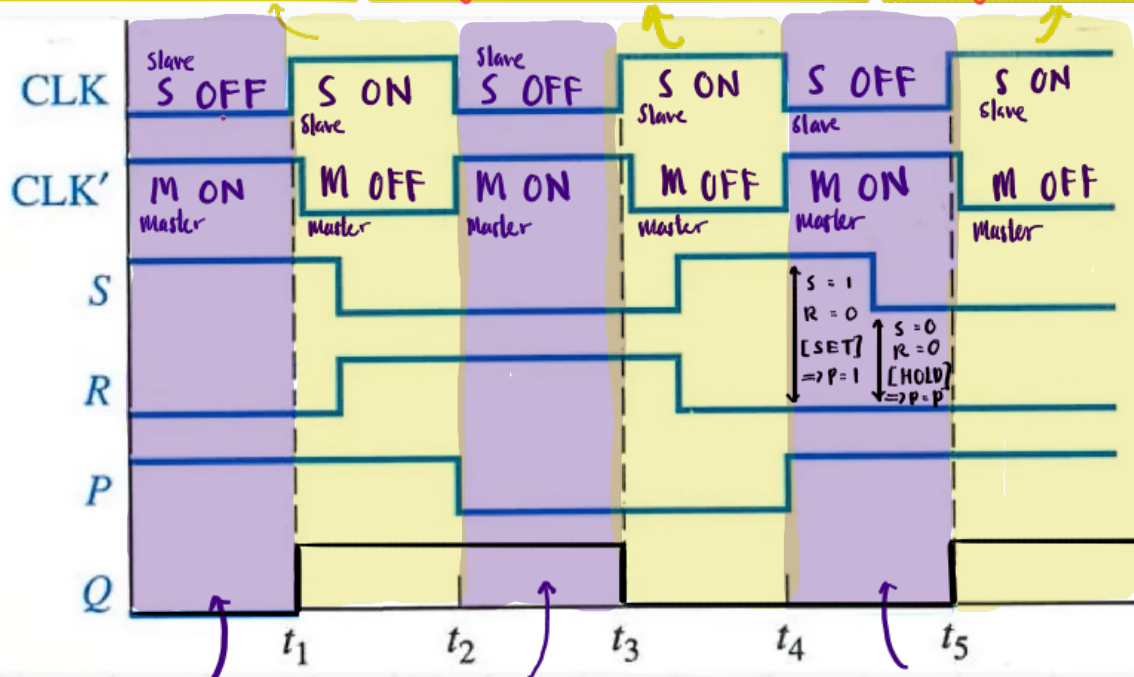
$\bullet N3 [P=1, CLK=1] = S_2 = 1$
 $\bullet N4 [P'=0, CLK=1] = R_2 = 0$
 Slave
 $\therefore$ SR Latch in SET action $[Q=1, Q'=0]$
 $\therefore$ No change in P cuz Master OFF

Master OFF, Slave ON

$\bullet N3 [P=0, CLK=1] = S_2 = 0$
 $\bullet N4 [P'=1, CLK=1] = R_2 = 1$
 Slave
 $\therefore$ SR Latch in RESET $[Q=0, Q'=1]$
 $\therefore$ No change in P cuz Master OFF

Master OFF, Slave ON

$\bullet N3 [P=1, CLK=1] = S_2 = 1$
 $\bullet N4 [P'=0, CLK=1] = R_2 = 0$
 Slave
 $\therefore$ SR Latch in SET $[Q=1, Q'=0]$
 $\therefore$ No change in P cuz Master OFF



Master ON, Slave OFF

$\bullet N1 [S=1, CLK'=1] \Rightarrow S_1 = 1$
 $\bullet N2 [R=0, CLK'=1] \Rightarrow R_1 = 0$
 Master
 $\therefore$ SR Latch in SET action $[P=1, P'=0]$
 $\therefore$ No change in Output [Q] cuz Slave OFF

Master ON, Slave OFF

$\bullet N1 [S=0, CLK'=1] \Rightarrow S_1 = 0$
 $\bullet N2 [R=1, CLK'=1] \Rightarrow R_1 = 1$
 Master
 $\therefore$ SR Latch in RESET $[P=0, P'=1]$
 $\therefore$ No change in Output [Q] cuz Slave OFF

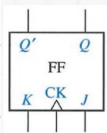
Master ON, Slave OFF

$\bullet N1 [S=0, CLK'=1] \Rightarrow S_1 = 0$
 $\bullet N2 [R=0, CLK'=1] \Rightarrow R_1 = 0$
 Master
 $\therefore$ SR Latch in HOLD $[P=P=1, P'=P'=0]$
 $\therefore$ No change in Output [Q] cuz Slave OFF

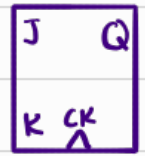
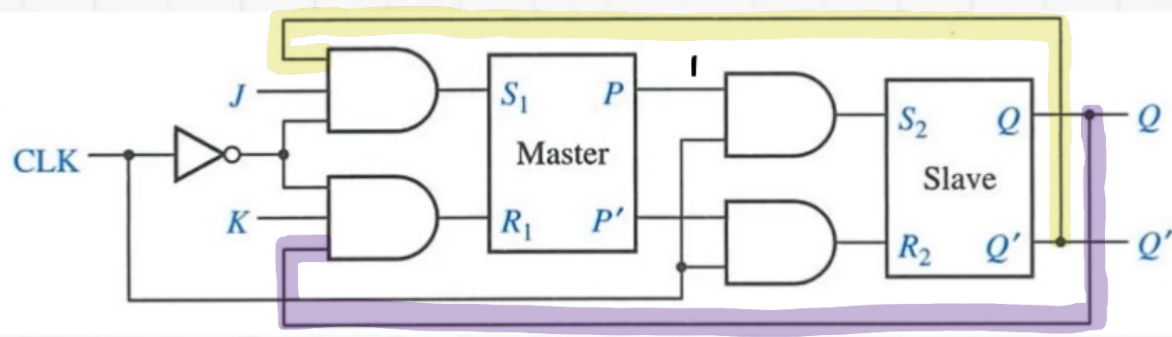
* Take Note : In this case is the positive Edge of the CLK [Posedge]
 Output [Q] only changes at CLK Edge. In other words Output only changes when CLK changes.

To make it negative-edge trigger, we just have to move the NOT gate to the Slave gate, so that a CLK change of $1 \rightarrow 0$, causes Slave to go $0 \rightarrow 1$, outputting/updating Q

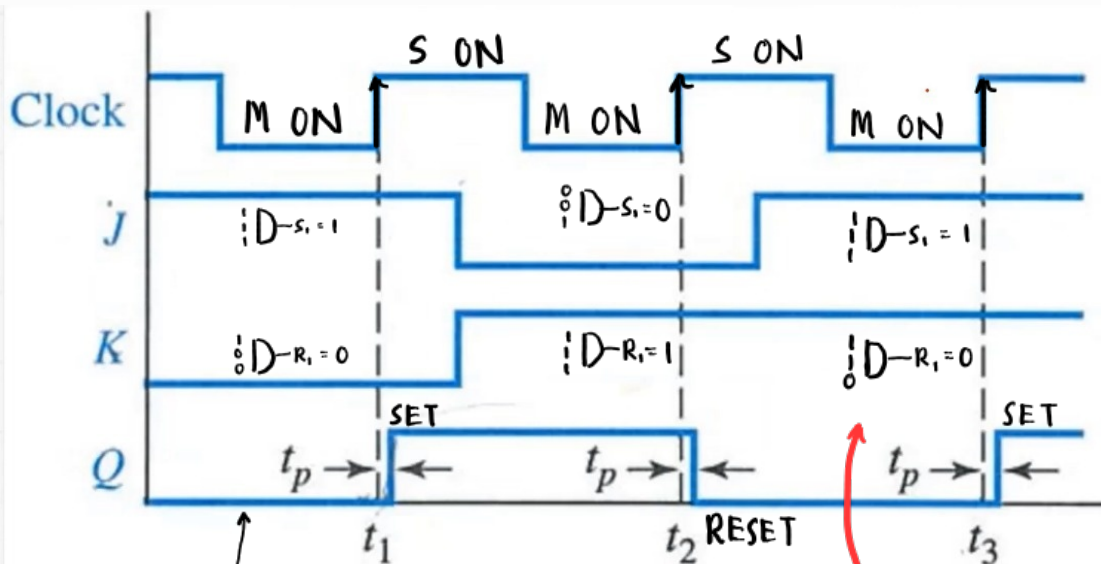
JK Flip-Flop



- Improved SR Flip-Flop [No more Forbidden case] [When $J=1, K=1$, Q is inverted the next cycle]



Using the same method to analyse this Master-Slave configuration, we get...

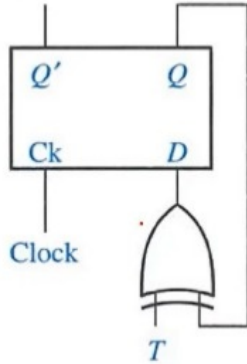
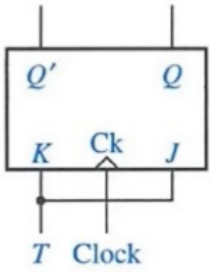
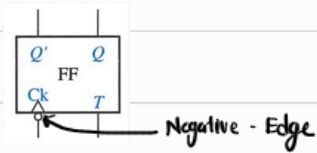


Assuming Q initially at 0

Notice that $J=1, K=1$ is not forbidden, because $Q=0$ is used as an input.

J	K	Q^+	Q'
0	0	Q HOLD	Q'
0	1	0 RESET	1
1	0	1 SET	0
1	1	Q' Toggle	Q

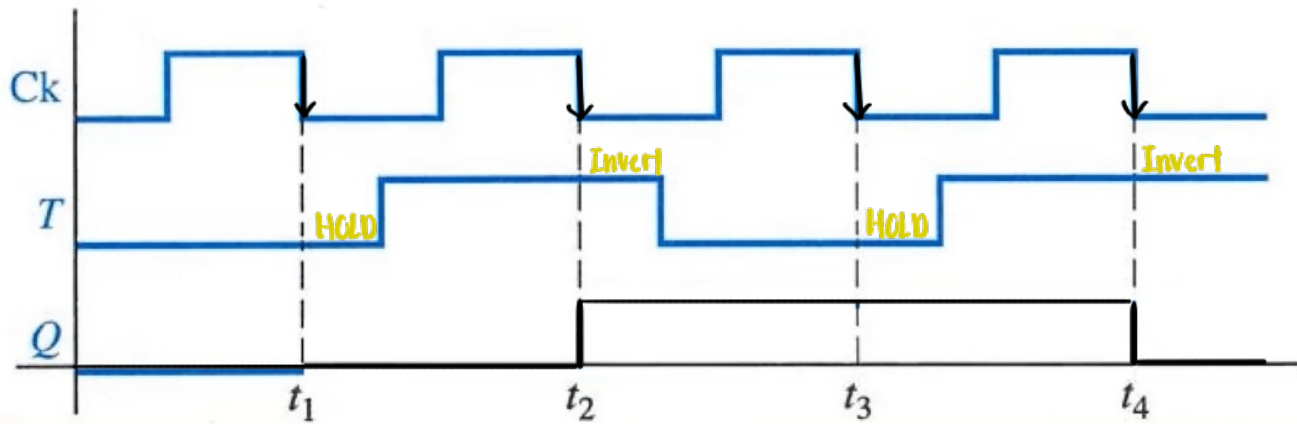
T Flip-Flop



	Latest Q	Updated Q
T	Q	Q ⁺
Hold	0	0
	0	1
Invert	1	1
	1	0

(a) Conversion of J-K to T

(b) Conversion of D to T



Designing Sequential Logic Design

In Combinational Logic Design, there are only 2 things

- Input we don't have current / Next state because combinational
- Output logic have no memory, so its impossible to have a current state

So we just draw a truth table, then a K-map with the inputs on the outside, outputs on the inside.

[Goal: Find simplest Boolean expression that directly produces the outputs from the inputs]

For Sequential Logic Design, we have

- Inputs [Optional]
- Current state
- Outputs [Optional]
- Next state

state transition table

1) So we draw a truth table w current & Next state,

2) Add the flip-flop required inputs to go from current $\rightarrow$ Next state in the state transition table

3) then a K-map for each flip-flop w input & current state on the outside,

^{excitation inputs} flip-flop required inputs on the inside [NOT the Next state]

[Goal: Find simplest Boolean expression for the signal that actually drive the flip-flops]

Example 1

Design a sequential logic design that generates the following output sequence using JK flip-flops

00, 01, 11, 10, 00

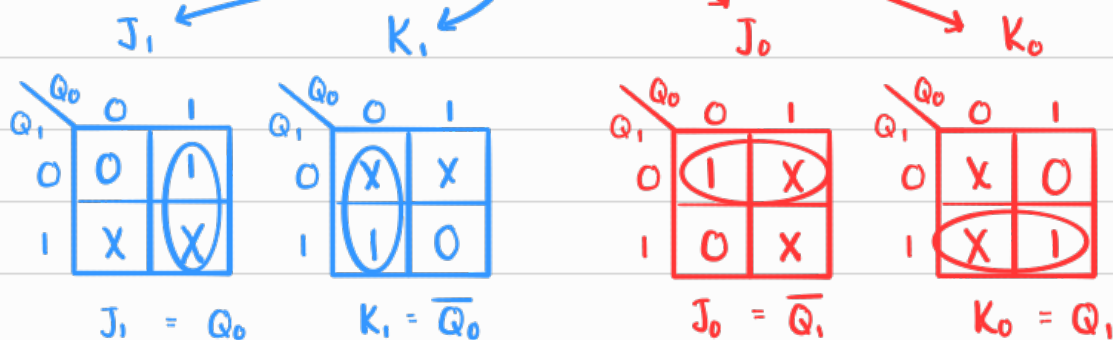
	J	K	Q^+
HOLD	0	0	Q
SET	1	0	1
RESET	0	1	0
Toggle	1	1	Q'

Since we need a 2-bit output, we need 2 JK flip-flops

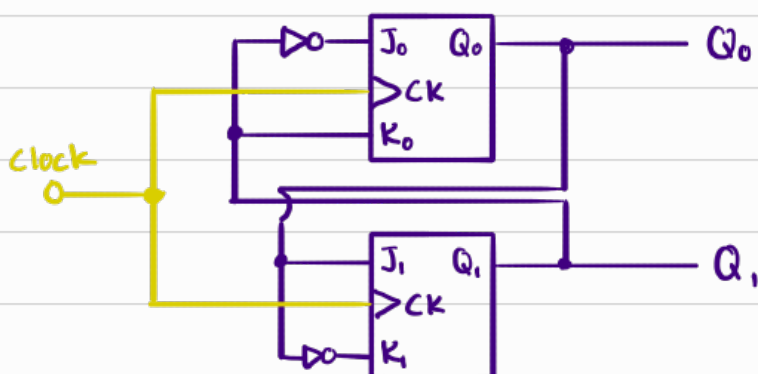
① State transition table

Current Q_1, Q_0	Next Q_1^+, Q_0^+	J, K	J_0, K_0
0 0	0 1	HOLD/RESET 0 X	SET / Toggle 1 X
0 1	1 1	SET / Toggle 1 X	HOLD / SET X 0
1 1	1 0	HOLD / SET X 0	RESET / Toggle X 1
1 0	0 0	RESET / Toggle X 1	HOLD / RESET 0 X

② K-maps



③ Implementation



Example 1

Design a 2-bit counter using 1 flip-flops. If input X is 0, counter increase by 1, while if the input X is 1, counter decrease by 1.

At pos-edge

T	Q^+
0	Q
1	Q'

HOLD
Toggle

Since we need a 2-bit output, we need 2 JK flip-flops

① State transition table

3 variable
Input $\therefore 2^3$ combination

X	$Q_1 Q_0$	$Q_1^+ Q_0^+$	T_1	T_0
0	0 0	0 1	0	1
1	0 0	1 1	1	1
0	0 1	1 0	1	1
1	0 1	0 0	0	1
0	1 0	1 1	0	1
1	1 0	0 1	1	1
0	1 1	0 0	1	1
1	1 1	1 0	0	1

② K-map

T_1

	$Q_1 Q_0$	00	01	11	10
X	0	0	1	1	0
1	1	1	0	0	1

$$T_1 = \bar{X}Q_0 + X\bar{Q}_0$$

$$T_1 = X \oplus Q_0$$

$$T_0 = 1$$

③ Implementation

